RUNGTA INTERNATIONAL SKILLS UNIVERSITY

RUNGTA INTERNATIONAL SKILLS UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY

# PROJECT REPORT

## CURRENCY CONVERTER

Project Serial Number: PS No. 35

| | |
|---|---|
| Student Name: | Sourya Agrawal |
| ERP / Roll Number: | RU-26-02439 |
| Course: | BCA (AI/ML) in Association with Google |
| Institution: | Rungta International Skills University |
| Submitted to: | Satyaprakash Sir |
| Academic Session: | 2026 – 2027 |

## 1. INTRODUCTION

The **Currency Converter** is a Python-based command-line application that converts an amount from one supported currency to another. It supports **INR, USD, EUR, and GBP**. The program uses a base-currency approach: it first converts the entered amount into INR (Indian Rupee) and then converts the INR value into the selected target currency.

## 2. OBJECTIVES

- To develop a simple and efficient currency conversion application using Python 3.
- To use a dictionary data structure to store supported currencies and sample exchange rates.
- To divide the program into modular, reusable functions (`convert_to_inr` and `convert_currency`).
- To validate user inputs, preventing crashes from negative amounts or non-numeric inputs.
- To display converted amounts cleanly rounded to two decimal places.

## 3. SOFTWARE AND HARDWARE REQUIREMENTS

**Software:** Python 3.x, Visual Studio Code / Pydroid 3 editor, and Terminal/Command Prompt.
**Hardware:** A computer or mobile device capable of executing Python 3 code.

## 4. SUPPORTED CURRENCIES AND RATES

The program uses the following illustrative exchange rates for demonstration, with INR as the base currency:

| Currency Code | Currency Name | Rate in INR (Base) |
|---|---|---|
| **INR** | Indian Rupee | 1 |
| **USD** | United States Dollar | 83 |
| **EUR** | Euro | 90 |
| **GBP** | British Pound | 105 |

## 5. WORKING PRINCIPLE / ALGORITHM

1. Display the supported currency codes (INR, USD, EUR, GBP).
2. Read user input for amount, source currency, and target currency.
3. Validate that the amount is a positive number and handle non-numeric errors using exception handling.
4. Check that both input currency codes exist in the `EXCHANGE_RATES` dictionary.
5. Convert source currency amount to base INR using `convert_to_inr()`.
6. Convert INR amount to target currency in `convert_currency()`.
7. Display the rounded output formatted to 2 decimal places.

## 6. PROGRAM STRUCTURE

**EXCHANGE_RATES:** Dictionary storing supported codes and INR-equivalent rates.
**convert_to_inr(amount, currency):** Multiplies amount by exchange rate.
**convert_currency(amount, from_curr, to_curr):** Calculates final converted amount.
**main():** Manages CLI workflow, validation, and error feedback.

## 7. SOURCE CODE

The complete Python source code for the project is included below (saved as `currency_converter.py`):

```
# Project: Currency Converter
# PS No. 35
# Language: Python 3

EXCHANGE_RATES = {
    "INR": 1,
    "USD": 83,
    "EUR": 90,
    "GBP": 105
}

def convert_to_inr(amount, currency):
    """Convert the given amount to INR."""
    return amount * EXCHANGE_RATES[currency]

def convert_currency(amount, from_currency, to_currency):
    """Convert an amount between supported currencies."""
    amount_in_inr = convert_to_inr(amount, from_currency)
    converted_amount = amount_in_inr / EXCHANGE_RATES[to_currency]
    return round(converted_amount, 2)

def main():
    print("===== CURRENCY CONVERTER =====")
    print("Supported currencies: INR, USD, EUR, GBP")

    try:
        amount = float(input("Enter amount: "))

        if amount < 0:
            print("Amount cannot be negative.")
            return

        from_currency = input("From currency: ").strip().upper()
        to_currency = input("To currency: ").strip().upper()

        if from_currency not in EXCHANGE_RATES:
            print(f"Unsupported currency '{from_currency}'. Available: INR, USD, EUR, GBP")
            return

        if to_currency not in EXCHANGE_RATES:
            print(f"Unsupported currency '{to_currency}'. Available: INR, USD, EUR, GBP")
            return

        result = convert_currency(amount, from_currency, to_currency)

        print(f"{amount:g} {from_currency} = {result:.2f} {to_currency}")

    except ValueError:
        print("Invalid amount. Enter a number.")

if __name__ == "__main__":
    main()
```

## 8. TESTING AND EXPECTED RESULTS

| Test | Input Scenario | Expected Output |
|---|---|---|
| 1 | 166, INR → USD | `166 INR = 2.00 USD` |
| 2 | 0, USD → INR | `0 USD = 0.00 INR` |
| 3 | 10, JPY → INR | Unsupported currency 'JPY' |
| 4 | "abc" as amount | Invalid amount. Enter a number. |
| 5 | -50 as amount | Amount cannot be negative. |
| 6 | 100, USD → EUR | `100 USD = 92.22 EUR` |

## EXECUTION SCREENSHOT (WINDOWS POWERSHELL)

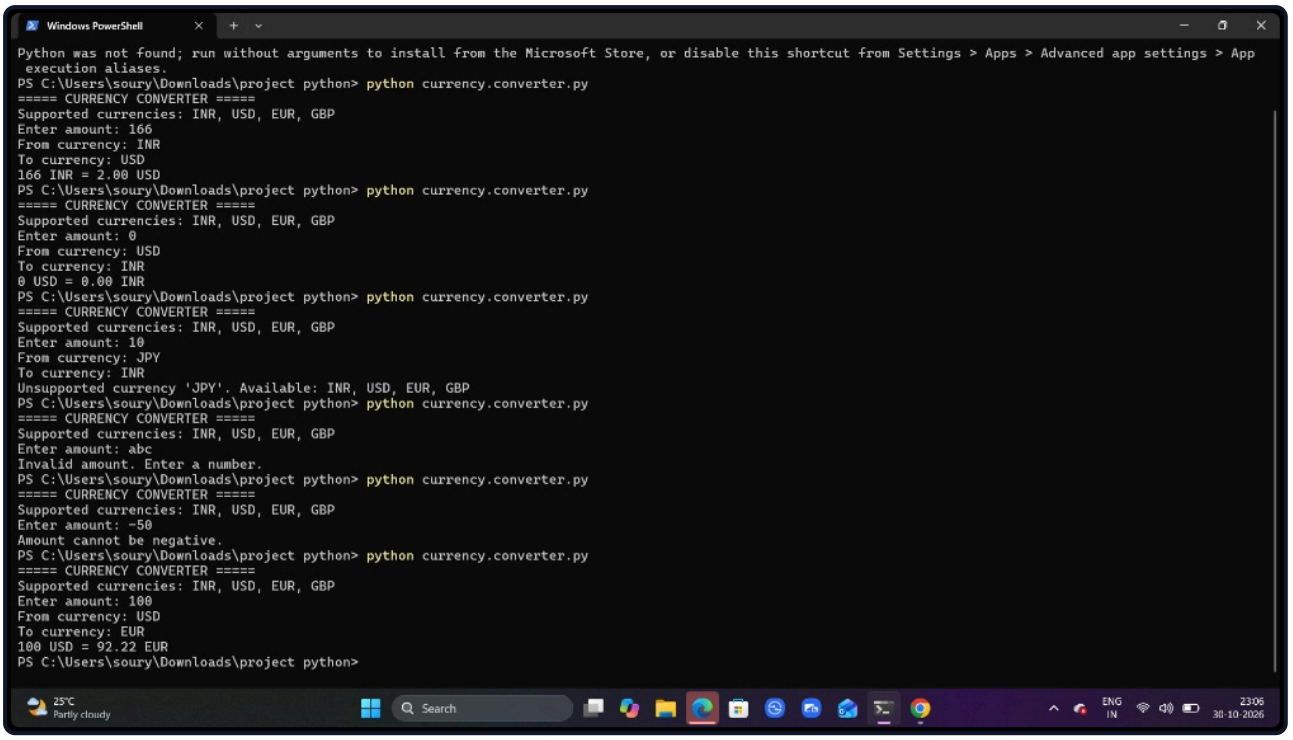



**Figure 1: Full-Resolution Test Case Execution Output in Windows PowerShell Terminal**

## 9. CONCLUSION

The Currency Converter project demonstrates effective use of Python dictionaries, modular functions, conditional logic, and exception handling. It reliably handles valid and invalid inputs across four currencies.

## 10. FUTURE SCOPE

Future improvements can include integration with live exchange rate APIs, adding a Graphical User Interface (GUI), supporting conversion logs, and automated unit testing.